

Data Analytics (DA)

Module-End -3

Module End Assignment: Screenshots

“Analyzing E-Learning Platform Purchases using MySQL” Assignment Task Overview

Task

1. Database Setup & Data Entry

- Create a new database.
- Create the three tables with proper:
 - Primary keys, Foreign keys, Appropriate data types
- Insert sample data:
 - 4–5 learners
 - 4–5 courses (different categories)
 - 6–8 purchase records

```
DROP DATABASE IF EXISTS Online_Course_DB;
```

```
CREATE DATABASE Online_Course_DB;
```

```
USE Online_Course_DB;
```

```
CREATE TABLE learners (
```

```
    learner_id INT PRIMARY KEY AUTO_INCREMENT,
```

```
    full_name VARCHAR(100) NOT NULL,
```

```
    country VARCHAR(50) NOT NULL
```

```
);
```

-- Courses table

```
CREATE TABLE courses (  
course_id INT PRIMARY KEY AUTO_INCREMENT,  
course_name VARCHAR(70) NOT NULL,  
category VARCHAR(50) NOT NULL,  
unit_price DECIMAL(10,2) NOT NULL  
);
```

-- purchase table

```
CREATE TABLE purchases(  
purchase_id INT PRIMARY KEY AUTO_INCREMENT,  
learner_id INT,  
course_id INT,  
quantity INT NOT NULL,  
purchase_date DATE NOT NULL,  
FOREIGN KEY (learner_id) REFERENCES learners(learner_id),  
FOREIGN KEY (course_id) REFERENCES courses(course_id)  
);
```

INSERT INTO learners (full_name, country)

VALUES

```
('Uma Senya ', 'India'),  
( 'Arun Kumar', 'India'),  
( 'Sarah Johnson', 'USA'),  
( 'David Smith', 'UK'),  
( 'Meera Patel', 'India');
```

INSERT INTO courses (course_name, category, unit_price)

VALUES

```
('Python for Data Analysis', 'Beginner', 2500.00),  
( 'SQL for Beginners', 'Beginner', 1800.00),  
( 'Power BI Dashboard', 'Intermediate', 3000.00),  
( 'Excel for Data Analysis', 'Beginner', 1500.00),  
( 'Machine Learning Basics', 'Advanced', 4000.00);
```

INSERT INTO purchases (learner_id, course_id, quantity, purchase_date)

VALUES

```
(1, 1, 1, '2026-01-10'),  
(1, 3, 1, '2026-02-15'),  
(2, 2, 2, '2026-01-20'),  
(3, 5, 1, '2026-03-05'),  
(4, 4, 3, '2026-02-10'),  
(5, 1, 1, '2026-03-15'),  
(3, 3, 2, '2026-04-01'),  
(2, 5, 1, '2026-04-10');
```

2. Data Exploration Using Joins

Use INNER, LEFT, and RIGHT JOIN to:

- Combine learner, course, and purchase data
- *Display:* Learner name, Course name, Category, Quantity, Total amount, Purchase date
- Presentation Requirements:
 - Format currency to 2 decimal places
 - Use column aliases
 - Sort by the highest total amount

File Edit View Query Database Server Tools Scripting Help

Navigator: sql assignment 2 Assignment 1 sql Module End Assignment-3

Schemas: Filter objects, companyinfo, sakila, sys, world

Administration Schemas

Information: Schema: sys

```
56 (2, 5, 1, '2026-04-10');
57
58 -- inner join
59 SELECT
60     l.full_name AS Learner_Name,
61     c.course_name AS Course_Name,
62     c.category AS Category,
63     p.quantity AS Quantity,
64     FORMAT(p.quantity * c.unit_price, 2) AS Total_Amount,
65     p.purchase_date AS Purchase_Date
```

Result Grid: Filter Rows: Export: Wrap Cell Contents: 15

Learner_Name	Course_Name	Category	Quantity	Total_Amount	Purchase_Date
Sarah Johnson	Power BI Dashboard	Intermediate	2	6,000.00	2026-04-01
David Smith	Excel for Data Analysis	Beginner	3	4,500.00	2026-02-10
Arun Kumar	Machine Learning Basics	Advanced	1	4,000.00	2026-04-10
Sarah Johnson	Machine Learning Basics	Advanced	1	4,000.00	2026-03-05
Arun Kumar	SQL for Beginners	Beginner	2	3,600.00	2026-01-20
Uma Senya	Power BI Dashboard	Intermediate	1	3,000.00	2026-02-15
Uma Senya	Python for Data Analysis	Beginner	1	2,500.00	2026-01-10
Meera Patel	Python for Data Analysis	Beginner	1	2,500.00	2026-03-15

Result 11 x

Output: Action Output

#	Time	Action	Message	Duration / Fetch
5	20:12:16	CREATE TABLE courses (course_id INT PRIMARY KEY AUTO_INCREMENT, course_name VARCHAR(70) ...	0 row(s) affected	0.032 sec
6	20:12:30	CREATE TABLE purchases(purchase_id INT PRIMARY KEY AUTO_INCREMENT, learner_id INT, course_id INT, ...	0 row(s) affected	0.094 sec
7	20:12:39	INSERT INTO learners (full_name, country) VALUES ('Uma Senya', 'India'), ('Arun Kumar', 'India'), ('Sarah Johnson', 'India'), ...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.015 sec
8	20:12:48	INSERT INTO courses (course_name, category, unit_price) VALUES ('Python for Data Analysis', 'Beginner', 2500.00), ...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.000 sec
9	20:12:55	INSERT INTO purchases (learner_id, course_id, quantity, purchase_date) VALUES (1, 1, 1, '2026-01-10'), (1, 1, 1, '2026-01-10'), (1, 1, 1, '2026-01-10'), ...	8 row(s) affected Records: 8 Duplicates: 0 Warnings: 0	0.031 sec
10	20:22:30	SELECT l.full_name AS Learner_Name, c.course_name AS Course_Name, c.category AS Category, ...	8 row(s) returned	0.000 sec / 0.000 sec

Object Info Session

File Edit View Query Database Server Tools Scripting Help

Navigator: sql assignment 2 Assignment 1 sql Module End Assignment-3

Schemas: Filter objects, companyinfo, sakila, sys, world

Administration Schemas

Information: Schema: sys

```
75 -- left join
76 -- Execute the selected portion of the script or everything, if there is no selection
77 SELECT
78     l.full_name AS Learner_Name,
79     c.course_name AS Course_Name,
80     c.category AS Category,
81     p.quantity AS Quantity,
82     FORMAT(p.quantity * c.unit_price, 2) AS Total_Amount,
83     p.purchase_date AS Purchase_Date
84 FROM learners l
```

Result Grid: Filter Rows: Export: Wrap Cell Contents: 15

Learner_Name	Course_Name	Category	Quantity	Total_Amount	Purchase_Date
Sarah Johnson	Power BI Dashboard	Intermediate	2	6,000.00	2026-04-01
David Smith	Excel for Data Analysis	Beginner	3	4,500.00	2026-02-10
Arun Kumar	Machine Learning Basics	Advanced	1	4,000.00	2026-04-10
Sarah Johnson	Machine Learning Basics	Advanced	1	4,000.00	2026-03-05
Arun Kumar	SQL for Beginners	Beginner	2	3,600.00	2026-01-20
Uma Senya	Power BI Dashboard	Intermediate	1	3,000.00	2026-02-15
Uma Senya	Python for Data Analysis	Beginner	1	2,500.00	2026-01-10
Meera Patel	Python for Data Analysis	Beginner	1	2,500.00	2026-03-15

Result 12 x

Output: Action Output

#	Time	Action	Message	Duration / Fetch
6	20:12:30	CREATE TABLE purchases(purchase_id INT PRIMARY KEY AUTO_INCREMENT, learner_id INT, course_id INT, ...	0 row(s) affected	0.094 sec
7	20:12:39	INSERT INTO learners (full_name, country) VALUES ('Uma Senya', 'India'), ('Arun Kumar', 'India'), ('Sarah Johnson', 'India'), ...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.015 sec
8	20:12:48	INSERT INTO courses (course_name, category, unit_price) VALUES ('Python for Data Analysis', 'Beginner', 2500.00), ...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.000 sec
9	20:12:55	INSERT INTO purchases (learner_id, course_id, quantity, purchase_date) VALUES (1, 1, 1, '2026-01-10'), (1, 1, 1, '2026-01-10'), (1, 1, 1, '2026-01-10'), ...	8 row(s) affected Records: 8 Duplicates: 0 Warnings: 0	0.031 sec
10	20:22:30	SELECT l.full_name AS Learner_Name, c.course_name AS Course_Name, c.category AS Category, ...	8 row(s) returned	0.000 sec / 0.000 sec
11	20:23:18	SELECT l.full_name AS Learner_Name, c.course_name AS Course_Name, c.category AS Category, ...	8 row(s) returned	0.015 sec / 0.000 sec

Object Info Session

File Edit View Query Database Server Tools Scripting Help

Navigator: sql assignment 2 Assignment 1 sql Module End Assignment-3

SCHEMAS

Filter objects

- companyinfo
- sakila
- sys
- world

Administration Schemas

Information:

Schema: sys

Result Grid

Learner_Name	Course_Name	Category	Quantity	Total_Amount	Purchase_Date
Sarah Johnson	Power BI Dashboard	Intermediate	2	6,000.00	2026-04-01
David Smith	Excel for Data Analysis	Beginner	3	4,500.00	2026-02-10
Sarah Johnson	Machine Learning Basics	Advanced	1	4,000.00	2026-03-05
Arun Kumar	Machine Learning Basics	Advanced	1	4,000.00	2026-04-10
Arun Kumar	SQL for Beginners	Beginner	2	3,600.00	2026-01-20
Uma Senya	Power BI Dashboard	Intermediate	1	3,000.00	2026-02-15
Uma Senya	Python for Data Analysis	Beginner	1	2,500.00	2026-01-10
Meera Patel	Python for Data Analysis	Beginner	1	2,500.00	2026-03-15

Result 27

Output

Action Output

#	Time	Action	Message	Duration / Fetch
22	20:44:07	WITH learner_spending AS (SELECT l.learner_id, l.full_name AS Learner_Name, l.country A...	5 row(s) returned	0.016 sec / 0.000 sec
23	08:00:13	SELECT c.course_name, COALESCE(COUNT(p.purchase_id), 0) AS purchase_count FROM courses c ...	5 row(s) returned	0.016 sec / 0.000 sec
24	08:03:07	CREATE VIEW category_performance_view AS SELECT c.category, SUM(p.quantity * c.unit_price) AS T...	0 row(s) affected	0.046 sec
25	08:03:07	SELECT * FROM category_performance_view LIMIT 0, 1000	3 row(s) returned	0.016 sec / 0.000 sec
26	08:05:04	SELECT * FROM category_performance_view LIMIT 0, 1000	3 row(s) returned	0.015 sec / 0.000 sec
27	09:32:45	SELECT l.full_name AS Learner_Name, c.course_name AS Course_Name, c.category AS Category, ...	8 row(s) returned	0.015 sec / 0.000 sec

Object Info Session

3. Core Analytical Queries (Q1-Q5)

Q1. Display each learner's total spending with their country.

File Edit View Query Database Server Tools Scripting Help

Navigator: sql assignment 2 Assignment 1 sql Module End Assignment-3

SCHEMAS

Filter objects

- companyinfo
- sakila
- sys
- world

Administration Schemas

Information:

Schema: sys

Result Grid

Learner_Name	Country	Total_Spending
Sarah Johnson	USA	10000.00
Arun Kumar	India	7600.00
Uma Senya	India	5500.00
David Smith	UK	4500.00
Meera Patel	India	2500.00

Result 14

Output

Action Output

#	Time	Action	Message	Duration / Fetch
8	20:12:48	INSERT INTO courses (course_name, category, unit_price) VALUES (Python for Data Analysis, 'Beginner', 25...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.000 sec
9	20:12:55	INSERT INTO purchases (learner_id, course_id, quantity, purchase_date) VALUES (1, 1, 1, 2026-01-10), (1...	8 row(s) affected Records: 8 Duplicates: 0 Warnings: 0	0.031 sec
10	20:22:30	SELECT l.full_name AS Learner_Name, c.course_name AS Course_Name, c.category AS Category, ...	8 row(s) returned	0.000 sec / 0.000 sec
11	20:22:18	SELECT l.full_name AS Learner_Name, c.course_name AS Course_Name, c.category AS Category, ...	8 row(s) returned	0.015 sec / 0.000 sec
12	20:24:40	SELECT l.full_name AS Learner_Name, c.course_name AS Course_Name, c.category AS Category, ...	8 row(s) returned	0.000 sec / 0.000 sec
13	20:25:37	SELECT l.full_name AS Learner_Name, l.country AS Country, SUM(p.quantity * c.unit_price) AS Total...	5 row(s) returned	0.000 sec / 0.000 sec

Object Info Session

Q2. Find the top 3 most purchased courses by quantity.

The screenshot shows a SQL IDE interface with a query editor and a result grid. The query is as follows:

```

129  SUM(p.quantity) AS Total_Quantity
130  FROM purchases p
131  INNER JOIN courses c
132  ON c.course_id = p.course_id
133  GROUP BY c.course_id, c.course_name
134  ORDER BY Total_Quantity DESC
135  LIMIT 3;

```

The result grid shows the following data:

Course_Name	Total_Quantity
Power BI Dashboard	3
Excel for Data Analysis	3
Python for Data Analysis	2

The Action Output section shows the execution of the query, with a message indicating that 8 rows were affected and 0 duplicates or warnings were found.

Q3. Show each category's:

- Total revenue
- Number of unique learners

The screenshot shows a SQL IDE interface with a query editor and a result grid. The query is as follows:

```

138  SELECT
139  SUM(p.quantity * c.unit_price) AS Total_Revenue,
140  COUNT(DISTINCT p.learner_id) AS Unique_Learners
141  FROM courses c
142  INNER JOIN purchases p
143  ON c.course_id = p.course_id
144  GROUP BY c.category
145  ORDER BY Total_Revenue DESC;

```

The result grid shows the following data:

Category	Total_Revenue	Unique_Learners
Beginner	13100.00	4
Intermediate	9000.00	2
Advanced	8000.00	2

The Action Output section shows the execution of the query, with a message indicating that 3 rows were returned.

Q4. List learners who purchased from more than one category.

The screenshot shows a database IDE with a SQL query editor and a results pane. The query is as follows:

```
-- Q4. List learners who purchased from more than one category
-- Execute the selected portion of the script or everything, if there is no selection
148 SELECT
149     1.full_name AS Learner_Name,
150     COUNT(DISTINCT c.category) AS Number_of_Categories
151 FROM learners l
152 INNER JOIN purchases p
153     ON l.learner_id = p.learner_id
154 INNER JOIN courses c
155     ON p.course_id = c.course_id
156 GROUP BY 1.learner_id, 1.full_name
157
```

The results pane shows the following data:

Learner_Name	Number_of_Categories
Uma Senya	2
Arun Kumar	2
Sarah Johnson	2

The bottom pane shows the action output log with the following entries:

#	Time	Action	Message	Duration / Fetch
11	20:23:18	SELECT	l.full_name AS Learner_Name, c.course_name AS Course_Name, c.category AS Category, ...	8 row(s) returned 0.015 sec / 0.000 sec
12	20:24:40	SELECT	l.full_name AS Learner_Name, c.course_name AS Course_Name, c.category AS Category, ...	8 row(s) returned 0.000 sec / 0.000 sec
13	20:25:37	SELECT	l.full_name AS Learner_Name, l.country AS Country, SUM(p.quantity * c.unit_price) AS Total_...	5 row(s) returned 0.000 sec / 0.000 sec
14	20:26:19	SELECT	c.course_name AS Course_Name, SUM(p.quantity) AS Total_Quantity FROM courses c INNER...	3 row(s) returned 0.016 sec / 0.000 sec
15	20:27:03	SELECT	c.category AS Category, SUM(p.quantity * c.unit_price) AS Total_Revenue, COUNT(DISTIN...	3 row(s) returned 0.016 sec / 0.000 sec
16	20:27:51	SELECT	l.full_name AS Learner_Name, COUNT(DISTINCT c.category) AS Number_of_Categories FRO...	3 row(s) returned 0.000 sec / 0.000 sec

Q5. Identify courses never purchased.

The screenshot shows a database IDE with a SQL query editor and a results pane. The query is as follows:

```
-- Q5. Identify courses never purchased
159 SELECT
160     c.course_name AS Course_Name,
161     c.category AS Category
162 FROM courses c
163 LEFT JOIN purchases p
164     ON c.course_id = p.course_id
165 WHERE p.purchase_id IS NULL;
166
167
```

The results pane shows the following data:

Course_Name	Category
-------------	----------

The bottom pane shows the action output log with the following entries:

#	Time	Action	Message	Duration / Fetch
12	20:24:40	SELECT	l.full_name AS Learner_Name, c.course_name AS Course_Name, c.category AS Category, ...	8 row(s) returned 0.000 sec / 0.000 sec
13	20:25:37	SELECT	l.full_name AS Learner_Name, l.country AS Country, SUM(p.quantity * c.unit_price) AS Total_...	5 row(s) returned 0.000 sec / 0.000 sec
14	20:26:19	SELECT	c.course_name AS Course_Name, SUM(p.quantity) AS Total_Quantity FROM courses c INNER...	3 row(s) returned 0.016 sec / 0.000 sec
15	20:27:03	SELECT	c.category AS Category, SUM(p.quantity * c.unit_price) AS Total_Revenue, COUNT(DISTIN...	3 row(s) returned 0.016 sec / 0.000 sec
16	20:27:51	SELECT	l.full_name AS Learner_Name, COUNT(DISTINCT c.category) AS Number_of_Categories FRO...	3 row(s) returned 0.000 sec / 0.000 sec
17	20:28:34	SELECT	c.course_name AS Course_Name, c.category AS Category FROM courses c LEFT JOIN purch...	0 row(s) returned 0.000 sec / 0.000 sec

4. Subqueries & Correlated Subqueries

Q6. Find learners whose total spending is above the average learner spending

The screenshot shows a database IDE with a SQL query editor and a results grid. The query is as follows:

```
-- Q6. Find learners whose total spending is above the average learner spending.
SELECT * FROM
(
    SELECT
        l.learner_id,
        l.full_name AS Learner_Name,
        SUM(p.quantity * c.unit_price) AS Total_Spending
    FROM learners l
    INNER JOIN purchases p
    ON l.learner_id = p.learner_id
)
```

The results grid shows the following data:

learner_id	Learner_Name	Total_Spending
2	Arun Kumar	7600.00
3	Sarah Johnson	10000.00

The output pane shows the execution log with the following messages:

- 13 20:25:37 SELECT l.full_name AS Learner_Name, l.country AS Country, SUM(p.quantity * c.unit_price) AS Total_... 5 row(s) returned
- 14 20:26:19 SELECT c.course_name AS Course_Name, SUM(p.quantity) AS Total_Quantity FROM courses c INNER... 3 row(s) returned
- 15 20:27:03 SELECT c.category AS Category, SUM(p.quantity * c.unit_price) AS Total_Revenue, COUNT(DISTIN... 3 row(s) returned
- 16 20:27:51 SELECT l.full_name AS Learner_Name, COUNT(DISTINCT c.category) AS Number_of_Categories FRO... 3 row(s) returned
- 17 20:28:34 SELECT c.course_name AS Course_Name, c.category AS Category FROM courses c LEFT JOIN purch... 0 row(s) returned
- 18 20:29:50 SELECT * FROM (SELECT l.learner_id, l.full_name AS Learner_Name, SUM(p.quantity * c.u... 2 row(s) returned

Q7. Display courses whose price is higher than any course in the 'Beginner' cate

The screenshot shows a database IDE with a SQL query editor and a results grid. The query is as follows:

```
-- Q7. Display courses whose price is higher than any course in the 'Beginner' category
SELECT
    course_name AS Course_Name,
    category AS Category,
    unit_price AS Unit_Price
FROM courses
WHERE unit_price > ANY
(
    SELECT unit_price
    FROM courses
    WHERE category = 'Beginner'
)
```

The results grid shows the following data:

Course_Name	Category	Unit_Price
Python for Data Analysis	Beginner	2500.00
SQL for Beginners	Beginner	1800.00
Power BI Dashboard	Intermediate	3000.00
Machine Learning Basics	Advanced	4000.00

The output pane shows the execution log with the following messages:

- 14 20:26:19 SELECT c.course_name AS Course_Name, SUM(p.quantity) AS Total_Quantity FROM courses c INNER... 3 row(s) returned
- 15 20:27:03 SELECT c.category AS Category, SUM(p.quantity * c.unit_price) AS Total_Revenue, COUNT(DISTIN... 3 row(s) returned
- 16 20:27:51 SELECT l.full_name AS Learner_Name, COUNT(DISTINCT c.category) AS Number_of_Categories FRO... 3 row(s) returned
- 17 20:28:34 SELECT c.course_name AS Course_Name, c.category AS Category FROM courses c LEFT JOIN purch... 0 row(s) returned
- 18 20:29:50 SELECT * FROM (SELECT l.learner_id, l.full_name AS Learner_Name, SUM(p.quantity * c.u... 2 row(s) returned
- 19 20:31:14 SELECT course_name AS Course_Name, category AS Category, unit_price AS Unit_Price FROM cour... 4 row(s) returned

Q8 . Find learners who spent more than the average spending in their country.

The screenshot shows a SQL query editor with the following code:

```

209 WHERE category = 'Beginner'
210 );
211
212 -- Q8 . Find learners who spent more than the average spending in their country.
213
214 SELECT
215     l.full_name AS Learner_Name,
216     l.country AS Country,
217     SUM(p.quantity * c.unit_price) AS Total_Spending
218 FROM learners l

```

The results grid shows the following data:

Learner_Name	Country	Total_Spending
Uma Senya	India	5500.00
Arun Kumar	India	7600.00

The bottom panel shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
15	20:27:03	SELECT	c.category AS Category, SUM(p.quantity * c.unit_price) AS Total_Revenue, COUNT(DISTINCT c.category) AS Number_of_Categories FROM courses c LEFT JOIN purchases p ON c.course_name = p.course_name	0.016 sec / 0.000 sec
16	20:27:51	SELECT	l.full_name AS Learner_Name, COUNT(DISTINCT c.category) AS Number_of_Categories FROM learners l LEFT JOIN purchases p ON l.learner_id = p.learner_id	0.000 sec / 0.000 sec
17	20:28:34	SELECT	c.course_name AS Course_Name, c.category AS Category FROM courses c LEFT JOIN purchases p ON c.course_name = p.course_name	0.000 sec / 0.000 sec
18	20:29:50	SELECT	FROM (SELECT l.learner_id, l.full_name AS Learner_Name, SUM(p.quantity * c.unit_price) AS Total_Spending FROM learners l LEFT JOIN purchases p ON l.learner_id = p.learner_id	0.000 sec / 0.000 sec
19	20:31:14	SELECT	course_name AS Course_Name, category AS Category, unit_price AS Unit_Price FROM courses c	0.016 sec / 0.000 sec
20	20:35:09	SELECT	l.full_name AS Learner_Name, l.country AS Country, SUM(p.quantity * c.unit_price) AS Total_Spending FROM learners l LEFT JOIN purchases p ON l.learner_id = p.learner_id	0.000 sec / 0.000 sec

5. CTE, CASE, View, and NULL Handling

Q9. Use a CTE to calculate total spending per learner, then:

Display learners with spending above 10,000

The screenshot shows a SQL query editor with the following code:

```

242
243 -- Q9. Use a CTE to calculate total spending per learner, then:
244 -- Display learners with spending above 10,000.
245
246 WITH learner_spending AS
247 (
248     SELECT
249         l.learner_id,
250         l.full_name AS Learner_Name,
251         l.country AS Country,

```

The results grid shows the following data:

learner_id	Learner_Name	Country	Total_Spending
------------	--------------	---------	----------------

The bottom panel shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
16	20:27:51	SELECT	l.full_name AS Learner_Name, COUNT(DISTINCT c.category) AS Number_of_Categories FROM learners l LEFT JOIN purchases p ON l.learner_id = p.learner_id	0.000 sec / 0.000 sec
17	20:28:34	SELECT	c.course_name AS Course_Name, c.category AS Category FROM courses c LEFT JOIN purchases p ON c.course_name = p.course_name	0.000 sec / 0.000 sec
18	20:29:50	SELECT	FROM (SELECT l.learner_id, l.full_name AS Learner_Name, SUM(p.quantity * c.unit_price) AS Total_Spending FROM learners l LEFT JOIN purchases p ON l.learner_id = p.learner_id	0.000 sec / 0.000 sec
19	20:31:14	SELECT	course_name AS Course_Name, category AS Category, unit_price AS Unit_Price FROM courses c	0.016 sec / 0.000 sec
20	20:35:09	SELECT	l.full_name AS Learner_Name, l.country AS Country, SUM(p.quantity * c.unit_price) AS Total_Spending FROM learners l LEFT JOIN purchases p ON l.learner_id = p.learner_id	0.000 sec / 0.000 sec
21	20:38:50	WITH learner_spending AS (SELECT l.learner_id, l.full_name AS Learner_Name, l.country AS Country, SUM(p.quantity * c.unit_price) AS Total_Spending FROM learners l LEFT JOIN purchases p ON l.learner_id = p.learner_id	0.000 sec / 0.000 sec	

Q10. CASE Expression

Classify learners based on spending:

- Above 15,000 → “High Value”,
- 8,000–15,000 → “Medium Value”,
- Below 8,000 → “Low Value”.

The screenshot shows a SQL IDE interface with a query editor and a result grid. The query is as follows:

```
WITH learner_spending AS
(
    SELECT
        l.learner_id,
        l.full_name AS Learner_Name,
        l.country AS Country,
        SUM(p.quantity * c.unit_price) AS Total_Spending
    FROM learners l
)
```

The result grid displays the following data:

Learner_Name	Country	Total_Spending	Learner_Category
Sarah Johnson	USA	10000.00	Medium Value
Arun Kumar	India	7600.00	Low Value
Uma Sanyal	India	5500.00	Low Value
David Smith	UK	4500.00	Low Value
Meera Patel	India	2500.00	Low Value

The bottom panel shows the execution log with the following actions:

#	Time	Action	Message	Duration / Fetch
17	20:28:34	SELECT	c.course_name AS Course_Name, c.category AS Category FROM courses c LEFT JOIN purch...	0 row(s) returned
18	20:29:50	SELECT * FROM (SELECT	l.learner_id, l.full_name AS Learner_Name, SUM(p.quantity * c.u...	2 row(s) returned
19	20:31:14	SELECT	course_name AS Course_Name, category AS Category, unit_price AS Unit_Price FROM cour...	4 row(s) returned
20	20:35:09	SELECT	l.full_name AS Learner_Name, l.country AS Country, SUM(p.quantity * c.unit_price) AS Total...	2 row(s) returned
21	20:38:50	WITH learner_spending AS (SELECT	l.learner_id, l.full_name AS Learner_Name, l.country A...	0 row(s) returned
22	20:44:07	WITH learner_spending AS (SELECT	l.learner_id, l.full_name AS Learner_Name, l.country A...	5 row(s) returned

Q11 . NULL Handling

- Display all courses and replace NULL purchase counts with 0 using: **IFNULL()** or **COALESCE()**

The screenshot shows a SQL IDE interface with a query editor and a result grid. The query is as follows:

```
-- Display all courses and replace courses with no purchases with 0.
SELECT
    c.course_name,
    COALESCE(COUNT(p.purchase_id), 0) AS purchase_count
FROM courses c
LEFT JOIN purchases p
ON c.course_id = p.course_id
GROUP BY c.course_id, c.course_name
```

The result grid displays the following data:

course_name	purchase_count
Python for Data Analysis	2
SQL for Beginners	1
Power BI Dashboard	2
Excel for Data Analysis	1
Machine Learning Basics	2

The bottom panel shows the execution log with the following actions:

#	Time	Action	Message	Duration / Fetch
18	20:29:50	SELECT * FROM (SELECT	l.learner_id, l.full_name AS Learner_Name, SUM(p.quantity * c.u...	2 row(s) returned
19	20:31:14	SELECT	course_name AS Course_Name, category AS Category, unit_price AS Unit_Price FROM cour...	4 row(s) returned
20	20:35:09	SELECT	l.full_name AS Learner_Name, l.country AS Country, SUM(p.quantity * c.unit_price) AS Total...	2 row(s) returned
21	20:38:50	WITH learner_spending AS (SELECT	l.learner_id, l.full_name AS Learner_Name, l.country A...	0 row(s) returned
22	20:44:07	WITH learner_spending AS (SELECT	l.learner_id, l.full_name AS Learner_Name, l.country A...	5 row(s) returned
23	08:00:13	SELECT	c.course_name, COALESCE(COUNT(p.purchase_id), 0) AS purchase_count FROM courses c ...	5 row(s) returned

Q12 . View

- Create a view: **category_performance_view**
- Showing:
 - *Category*
 - *Total revenue*
 - *Number of purchases*
 - *Average revenue per purchase*

The screenshot displays the SQL Developer interface. The main window shows a SQL script for creating a view and querying it. The script is as follows:

```
316 SUM(p.quantity * c.unit_price) AS total_revenue,  
317 Execute the selected portion of the script or everything, if there is no selection  
318 AVG(p.quantity * c.unit_price) AS average_revenue_per_purchase  
319 FROM courses c  
320 JOIN purchases p  
321 ON c.course_id = p.course_id  
322 GROUP BY c.category;  
323  
324 SELECT * FROM category_performance_views;  
325
```

Below the script, the 'Result Grid' shows the output of the query:

category	total_revenue	number_of_purchases	average_revenue_per_purchase
Beginner	13100.00	4	3275.000000
Intermediate	9000.00	2	4500.000000
Advanced	8000.00	2	4000.000000

The bottom pane shows the 'Output' window with a log of database actions:

#	Time	Action	Message	Duration / Fetch
21	20:38:50	WITH learner_spending AS (SELECT l.learner_id, l.full_name AS Learner_Name, l.country A...	0 row(s) returned	0.000 sec / 0.000 sec
22	20:44:07	WITH learner_spending AS (SELECT l.learner_id, l.full_name AS Learner_Name, l.country A...	5 row(s) returned	0.016 sec / 0.000 sec
23	08:00:13	SELECT c.course_name, COALESCE(COUNT(p.purchase_id), 0) AS purchase_count FROM courses c ...	5 row(s) returned	0.016 sec / 0.000 sec
24	08:03:07	CREATE VIEW category_performance_view AS SELECT c.category, SUM(p.quantity * c.unit_price) AS t...	0 row(s) affected	0.046 sec
25	08:03:07	SELECT * FROM category_performance_view LIMIT 0, 1000	3 row(s) returned	0.016 sec / 0.000 sec
26	08:05:04	SELECT * FROM category_performance_view LIMIT 0, 1000	3 row(s) returned	0.015 sec / 0.000 sec

Summary Report

Key Insights from the Analysis

- Beginner courses generated the highest overall revenue
- Power BI Dashboard was the most frequently purchased courses, indicating strong learner interest in intermediate-level data analytics skills.
- Some learners purchased courses from more than one category, showing interest in expanding their knowledge across different skill levels.
- Learner spending varied significantly, allowing learners to be classified as High Value, Medium Value, or Low Value based on total spending.
- The analysis compared individual learner spending with both the overall average spending and the average

spending within each learner's country.

- Course categories were evaluated using total revenue, number of purchases, and average revenue per purchase through the category_performance_view.
- The analysis also included identification of courses with no purchases, helping monitor potential underperforming courses.

Brief Recommendations

- Focus marketing efforts on top-performing categories and popular courses to increase revenue and attract more learners.
- Promote bundle offers across Beginner, Intermediate, and Advanced courses to encourage purchases from multiple categories.
- Target high-spending and medium-spending learners with personalized course recommendations and advanced learning opportunities.
- Review courses with low or no purchases and consider improving their content, pricing, or promotional strategy.
- Use the category performance view regularly to monitor revenue, purchase activity, and average revenue per purchase.
- Encourage progression between course levels by recommending Intermediate courses to learners who complete Beginner courses.

Conclusion

This analysis demonstrates how SQL can transform online course purchase data into meaningful business insights. By analyzing learner spending, course demand, category revenue, and purchasing patterns, the organization can make more informed decisions regarding marketing strategies, course development, and learner engagement.